

Week 4 – Master Study Notes

SPARQL: Querying RDF Data & Triple Stores

F20BD / F21BD – Big Data Management

Heriot-Watt University

Class Test Scope – READ FIRST

The **Week 7 class test** covers:

- Introduction to Big Data
- RDF / RDFS
- **SPARQL ← THIS LECTURE – heavily tested!**
- SKOS

NOT included in class test: OWL / Protégé (but **IS in the final exam**).

Throughout these notes, anything highlighted in **purple boxes** is **directly class-test relevant**.

Contents

1	The Big Picture – Where Does SPARQL Fit?	2
1.1	The Full Pipeline: From Data to Answers	2
2	Triple Stores – Where RDF Lives	2
2.1	How Can RDF Be Stored?	2
2.2	Apache Jena Framework	3
2.3	Fuseki – Your SPARQL Server	3
2.4	SPARQL Endpoints	3
2.5	ETL: Populating a Triplestore	3
3	SPARQL Fundamentals	4
3.1	The Core Idea: Pattern Matching	4
3.2	SPARQL Variables	5
4	SPARQL Query Structure	6
4.1	Triple Patterns Use Turtle Syntax	7
5	The Four SPARQL Query Types	8
5.1	SELECT – The Most Common	8
5.2	CONSTRUCT – Build New Triples	9
5.3	DESCRIBE – Tell Me Everything About X	9
5.4	ASK – Yes or No?	9
6	OPTIONAL – Left Joins in SPARQL	11
7	FILTER – Restricting Results	11
7.1	Filtering Numbers	12
7.2	Filtering Dates	12
7.3	Filtering Strings with REGEX	12
7.4	Negation: NOT EXISTS and MINUS	12
8	UNION – Combining Alternative Patterns	13

9 Query Modifiers: ORDER BY, LIMIT, OFFSET	14
10 GROUP BY, HAVING, and Aggregate Functions	14
11 SPARQL 1.1 Features (2013)	16
11.1 Property Paths – Transitive Traversal	16
11.2 Subqueries	16
11.3 Summary: SPARQL 1.1 Additions	17
12 SERVICE – Federated Queries	17
13 Reasoning: Forward and Backward Chaining	18
14 Practice Questions – Class Test Style	20
14.1 Q1: Reading a SPARQL Query	20
14.2 Q2: OPTIONAL vs Required	20
14.3 Q3: Identify the Query Type	20
14.4 Q4: FILTER vs HAVING	20
14.5 Q5: Property Paths	21
14.6 Q6: Forward vs Backward Chaining	21
14.7 Q7: Writing a SPARQL Query from English	21
14.8 Q8: Pagination	21
15 Quick Reference Cheat Sheet	22

1 The Big Picture – Where Does SPARQL Fit?

Recap: The Data Pipeline

In Week 2, you learned how to **represent** knowledge using RDF triples. But what's the point of storing all that data if you can't *ask questions* about it?

SPARQL is the language that lets you **query** (search) RDF data. It's the SQL of the Semantic Web.

Think of it this way:

- **RDF** = the data (facts stored as triples)
- **RDFS/OWL** = the rules (what the data means)
- **SPARQL** = the questions ("find me all people who know Bob")

1.1 The Full Pipeline: From Data to Answers

2 Triple Stores – Where RDF Lives

Know What a Triplestore Is

You should understand what a triplestore is, what Jena/Fuseki are, and the concept of a SPARQL endpoint.

What is a Triplestore?

A **triplestore** (also called an RDF store) is a database specifically designed to store and retrieve **RDF triples** (subject-predicate-object).

Analogy: If RDF triples are books, a triplestore is the library. And SPARQL is the librarian you ask to find books.

Key capabilities:

- Store millions/billions of RDF triples
- Manage **multiple datasets**, each with one or more **named graphs**
- Accept and execute **SPARQL queries**
- Perform **reasoning/inference** (built-in or via external reasoners)
- Query across multiple graphs at once

2.1 How Can RDF Be Stored?

RDF is a graph data model, but it can be stored in different technologies:

Storage Type	How It Works	Examples
Native Triplestore	Purpose-built for RDF triples. Best performance for graph queries.	Stardog, Neptune, GraphDB
Relational DB	Map triples to SQL tables (e.g., one big 3-column table). Works but slower for graph traversal.	Jena SDB, Virtuoso
NoSQL	Store triples in document or key-value stores.	MongoDB (custom mapping)

2.2 Apache Jena Framework

Jena = The Swiss Army Knife for RDF

Apache Jena is a free, open-source **Java framework** for building Semantic Web applications. It provides:

API	What It Does
RDF API	Create, read, manipulate RDF triples in Java
Ontology API	Work with RDFS and OWL ontologies
SPARQL API	Run SPARQL queries programmatically
Inference API	Built-in and external reasoners for deriving new facts
Store API	In-memory, TDB (native), or relational backends
Parsers/Writers	Read/write RDF/XML, Turtle, N-Triples, RDFa

2.3 Fuseki – Your SPARQL Server

Fuseki

Fuseki is a **SPARQL server** built on top of Jena. Think of it as a web application that:

- Runs on your computer (localhost:3030)
- Stores your RDF datasets in TDB2 (Jena's native triplestore)
- Provides a **web interface** to upload data, run queries, and view results
- Exposes **SPARQL endpoints** so other applications can query your data over HTTP

Analogy: Fuseki is like a web-based database management tool (like phpMyAdmin for MySQL), but for RDF data.

2.4 SPARQL Endpoints

SPARQL Endpoint

A **SPARQL endpoint** is a **web service** (URL) that accepts SPARQL queries over HTTP and returns results.

Analogy: It's like a restaurant that takes orders (SPARQL queries) and serves food (results).

Examples of public endpoints:

- <http://dbpedia.org/sparql> – query Wikipedia's structured data
- <http://localhost:3030/dataset/sparql> – your local Fuseki endpoint

Results can be returned as XML, JSON, CSV, or other formats.

2.5 ETL: Populating a Triplestore

ETL = Extract, Transform, Load

Creating triples manually is too slow. In practice, automated tools follow an **ETL** pattern:

1. **Extract** – Pull data from databases, CSV files, semi-structured data, text, etc.
2. **Transform** – Map the data to the ontology (convert to RDF triples using IRIs)
3. **Load** – Insert the triples into the triplestore

Analogy: ETL is like taking all your messy handwritten notes (raw data), typing them into a standard format (transform), and filing them in the library catalogue (load).

3 SPARQL Fundamentals

SPARQL is CORE to the class test

You MUST know: what SPARQL is, pattern matching, variables, the four query types (SELECT, CONSTRUCT, DESCRIBE, ASK), query structure (PREFIX, SELECT, WHERE), and how to read/write basic SPARQL queries.

What is SPARQL?

SPARQL (pronounced “sparkle”) stands for **SPARQL Protocol and RDF Query Language**. It is to RDF what **SQL is to relational databases** – the standard way to ask questions about your data.

The **SPARQL** standard has 4 parts:

Part	What It Does
Query Language	Retrieve data (SELECT, CONSTRUCT, DESCRIBE, ASK)
Protocol	Send queries to endpoints over HTTP
Query Results Format	Format of returned data (XML, JSON, CSV)
Update Language	Add/modify/delete triples (INSERT, DELETE)

3.1 The Core Idea: Pattern Matching

SPARQL = Pattern Matching

SPARQL works by **pattern matching**. You describe a “shape” (pattern) that you want to find in the graph, and the engine finds all parts of the graph that match.

Step-by-step:

1. **You** describe the pattern you want (your SPARQL query)
2. The **SPARQL engine** searches the entire RDF graph
3. The **results** are all parts of the graph that match your pattern

Example:

Pattern Matching – Finding All People **Goal:** Find all people in the graph.

Pattern: “Find anything (?p) that is a type of foaf:Person”

```
SELECT ?p
WHERE {
  ?p rdf:type foaf:Person .
}
```

What this means in English:

- ?p is a **variable** – “I don’t know what it is yet, find it for me”
- **rdf:type** means “is a type of”
- **foaf:Person** is the class we’re looking for
- The engine searches every triple and returns all ?p values where this pattern matches

3.2 SPARQL Variables

Variables Start with ? or \$

In SPARQL, **variables** are placeholders that the engine fills in with matching values.

- Variables start with ? or \$ – both are valid and identical
- ?name and \$name are the same variable
- ? is used 99% of the time in practice
- Variables can appear in the subject, predicate, AND/OR object positions

Any Position Can Be a Variable

Pattern	What It Finds	Positions
?s ?p ?o	ALL triples in the graph	All three are variables
?s prt:name ?name	All things with a name	Subject + object variable
prt:Au ?p ?o	All properties of gold (Au)	Predicate + object variable
?s prt:symbol "Au"	The element with symbol "Au"	Only subject is variable

4 SPARQL Query Structure

Know the Query Structure

You MUST be able to identify and write the correct order of clauses in a SPARQL query.

The Anatomy of a SPARQL Query

Every SPARQL query follows this structure:

```

PREFIX foaf: <http://xmlns.com/foaf/0.1/>      (*@\textcolor{exgreen}{\
    textbf{1. Prefixes -- namespace shortcuts}}@*)
PREFIX ex:   <http://example.org/>

SELECT ?name ?age                               (*@\textcolor{exgreen}{\
    textbf{2. Query type + variables to return}}@*)
FROM <http://example.org/mygraph>                (*@\textcolor{exgreen}{\
    textbf{3. Which graph (optional)}}@*)
WHERE {                                           (*@\textcolor{exgreen}{\
    textbf{4. Patterns to match}}@*)
    ?person foaf:name ?name .
    ?person foaf:age ?age .
    FILTER(?age > 18)
}
ORDER BY ?name                                   (*@\textcolor{exgreen}{\
    textbf{5. Modifiers (optional)}}@*)
LIMIT 10
  
```

#	Clause	Purpose
1	PREFIX	Define namespace shortcuts (like @prefix in Turtle)
2	SELECT / CONSTRUCT / DESCRIBE / ASK	The query type + which variables to display
3	FROM	Which named graph(s) to query (optional – defaults to default graph)
4	WHERE { ... }	Graph patterns to match, written in Turtle-like syntax
5	Modifiers	ORDER BY, LIMIT, OFFSET, GROUP BY, HAVING

PREFIX Must Come FIRST PREFIX declarations ALWAYS come before everything else. They define the namespace shortcuts used throughout the query. In SPARQL, you write PREFIX (without the @ and without the trailing dot that Turtle uses).

Turtle: @prefix foaf: <http://xmlns.com/foaf/0.1/> .

SPARQL: PREFIX foaf: <http://xmlns.com/foaf/0.1/> (no @ and no dot!)

4.1 Triple Patterns Use Turtle Syntax

△ Warning:

SPARQL WHERE = Turtle Inside! The patterns inside WHERE { } use **Turtle syntax**. This means:

- Each triple ends with a **period** (.)
- **Semicolons** (;) = same subject, new predicate
- **Commas** (,) = same subject + predicate, new object
- **a** = shortcut for `rdf:type`
- IRIs in `<angle brackets>` or QNames like `foaf:Person`
- Literals in "double quotes" with optional `^^datatype`

If you learned Turtle from Week 2, you already know how to write SPARQL patterns!

Example:

Turtle Shortcuts Inside SPARQL These two queries are **identical**:

Without shortcuts (verbose):

```
SELECT ?e ?c ?a WHERE {
  ?e prt:name "gold" .
  ?e prt:classification ?c .
  ?e prt:atomicNumber ?a .
}
```

With semicolons (compact):

```
SELECT ?e ?c ?a WHERE {
  ?e prt:name "gold" ;
    prt:classification ?c ;
    prt:atomicNumber ?a .
}
```

The semicolons mean “same subject (?e), different predicate” – exactly like Turtle!

Example:

Four Ways to Write the Same Query Find the name of the element whose IRI is “Au”:

```
(*@\textcolor{defblue}{1.}*) SELECT ?name WHERE { <http://.../Au> <http://...#name> ?name }
(*@\textcolor{defblue}{2.}*) SELECT ?name WHERE { prt:Au prt:name ?name }
(*@\textcolor{defblue}{3.}*) SELECT ?name WHERE { ?e prt:symbol "Au" .
  ?e prt:name ?name }
(*@\textcolor{defblue}{4.}*) SELECT ?name WHERE { ?e prt:symbol "Au" ;
  prt:name ?name }
```

All four return the same result! Query 1 uses full IRIs. Query 2 uses QNames. Queries 3 and 4 find the element by symbol first. Query 4 uses the semicolon shortcut.

5 The Four SPARQL Query Types

Must Know All Four Query Types

The exam will ask you which query type returns what. You **MUST** know SELECT, CONSTRUCT, DESCRIBE, and ASK – what each returns and when to use each.

SELECT

Returns a table
of variable bindings

CONSTRUCT

Returns a new
RDF graph (triples)

DESCRIBE

Returns all triples
about a resource

ASK

Returns true/false
(does pattern exist?)

5.1 SELECT – The Most Common

SELECT Returns a Table

SELECT is the most common query type. It returns a **table** where:

- Each **column** is a variable from the SELECT clause
- Each **row** is a set of values that matched the pattern

It's almost identical to SQL SELECT.

Example:

SELECT All Element Names

```
PREFIX prt: <http://www.daml.org/2003/01/periodictable/PeriodicTable#>
SELECT ?name
WHERE { ?element prt:name ?name . }
```

Result: A table with one column (?name) showing the name of every element:

?name
"Hydrogen"
"Helium"
"Lithium"
...

Note: Only ?name appears in the results because only ?name is in the SELECT clause. ?element was used in the pattern but not selected for display.

Tip:

SELECT * Returns Everything SELECT * returns **all variables** used in the WHERE clause.
SELECT DISTINCT removes duplicate rows.

5.2 CONSTRUCT – Build New Triples

CONSTRUCT Returns an RDF Graph

CONSTRUCT creates **new triples** from a template. For every match in the WHERE clause, it generates triples using the CONSTRUCT template.

Use case: Transforming data from one vocabulary to another.

Example:

CONSTRUCT – Vocabulary Transformation **Goal:** Take element data and re-express it using `rdfs:label` and `rdfs:comment`:

```
PREFIX prt: <http://...>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

CONSTRUCT {
  ?element rdfs:label ?name ;
           rdfs:comment ?color .
}
WHERE {
  ?element prt:name ?name ;
           prt:symbol ?symbol .
  OPTIONAL { ?element prt:color ?color }
}
```

Result: A new RDF graph with triples like:

```
prt:Chlorine rdfs:label "Chlorine" ;
            rdfs:comment "greenish yellow" .
prt:Hydrogen rdfs:label "Hydrogen" .
```

Note: Hydrogen has no `rdfs:comment` because it had no colour data (OPTIONAL was used).

5.3 DESCRIBE – Tell Me Everything About X

DESCRIBE Returns All Properties

DESCRIBE retrieves **all triples** where a resource appears as the subject – effectively dumping everything the graph knows about it.

Two forms:

- DESCRIBE `prt:Au` – describe one specific resource
- DESCRIBE `?e WHERE { ?e a prt:Element }` – find resources matching a pattern, then describe them all

5.4 ASK – Yes or No?

ASK Returns true/false

ASK checks whether a pattern has **any matches at all**. It returns:

- **true** if at least one match exists
- **false** if zero matches exist

It does NOT return the matching data – just yes or no.

Example:

ASK – Does An Element Symbol Contain 'A'?

```
PREFIX prt: <http://...>
```

```
ASK {
  ?e a prt:Element ;
    prt:symbol ?s .
  FILTER REGEX(?s, 'A', 'i')
}
```

Result: true (because elements like Ag, Al, Ar, Au, etc. exist).

When to Use Which Query Type?

Need	Use	Example
Get a table of results	SELECT	“List all element names and symbols”
Create new RDF triples	CONSTRUCT	“Convert data from one vocabulary to another”
Get everything about a resource	DESCRIBE	“Show me all properties of gold”
Check if something exists	ASK	“Is there an element with symbol ‘Zz’?”

Which Query Types Return RDF Triples (Graphs)?

Query Type	Returns Table?	Returns Graph (Triples)?
SELECT	✓ Yes	✗ No
CONSTRUCT	✗ No	✓ Yes
DESCRIBE	✗ No	✓ Yes
ASK	✗ No (returns boolean)	✗ No

Memory trick: CONSTRUCT and DESCRIBE both return **graphs**. SELECT returns a **table**. ASK returns **true/false**.

6 OPTIONAL – Left Joins in SPARQL

OPTIONAL is Frequently Tested

Know what OPTIONAL does, how it differs from a required pattern, and that it acts like a SQL LEFT JOIN.

OPTIONAL = “Include If Available”

OPTIONAL means: try to match this pattern, but if it doesn't match, **still include the result** with blank/null values for the missing variables.

Without OPTIONAL: Missing data = row excluded entirely.

With OPTIONAL: Missing data = row included with blanks.

Analogy: “Give me everyone's name. If they have an email, show that too, but don't skip people just because they lack an email.”

Example:

OPTIONAL vs Required – 118 Elements, 3 Have No Colour **Query A (required):**

```
SELECT ?name ?color WHERE {
  ?e prt:name ?name ; prt:color ?color .
}
```

Result: 115 rows (3 elements without colour are **excluded**).

Query B (optional):

```
SELECT ?name ?color WHERE {
  ?e prt:name ?name .
  OPTIONAL { ?e prt:color ?color }
}
```

Result: 118 rows (all elements included; 3 have blank colour).

OPTIONAL = SQL LEFT JOIN This is a common exam comparison:

- Required pattern = SQL **INNER JOIN** (both sides must match)
- OPTIONAL pattern = SQL **LEFT JOIN** (keep left side even if right has no match)

Multiple OPTIONALS are valid and each one independently says “include if available.”

7 FILTER – Restricting Results

FILTER is Heavily Tested

Know: numeric filters, date filters, REGEX for strings, FILTER NOT EXISTS, FILTER EXISTS, and MINUS.

FILTER = Boolean Conditions

FILTER restricts which matches are returned by applying a **boolean condition**. If the condition is true, the match is kept; if false, it's discarded.

Operators: >, <, >=, <=, =, !=, && (AND), || (OR), ! (NOT)

7.1 Filtering Numbers

Example:

Find Elements with Atomic Number < 10

```
SELECT ?symbol ?number ?state WHERE {
  ?element prt:symbol ?symbol ;
           prt:atomicNumber ?number ;
           prt:standardState ?state .
  FILTER(?number < 10)
}
```

Returns only the first 9 elements (H through F).

7.2 Filtering Dates

Example:

Find People Born Between 1962 and 2020

```
SELECT * WHERE {
  ?sub pep:bornOn ?date .
  FILTER(?date > "1962" && ?date < "2020")
}
```

The && means AND – both conditions must be true. Date comparisons use the same operators as numbers.

7.3 Filtering Strings with REGEX

REGEX for String Matching

`FILTER REGEX(variable, pattern, flags)` tests strings against a regular expression.

Argument	What It Is	Example
variable	The string to test	<code>?symbol</code>
pattern	The regex pattern	<code>'^n'</code> (starts with n)
flags	Optional modifiers	<code>'i'</code> = case-insensitive

Common patterns: `^` = start of string, `$` = end of string, `.` = any character, `*` = zero or more.

Example:

Find Symbols Starting with N (Case-Insensitive)

```
FILTER REGEX(?symbol, '^n', 'i')
```

Matches: Na, Ne, Ni, Nb, Nd, etc. The `'i'` flag makes it match both `'n'` and `'N'`.

7.4 Negation: NOT EXISTS and MINUS

Finding Things That DON'T Match

Two ways to find “rows where something is absent”:

Keyword	How It Works
FILTER NOT EXISTS { pattern }	Keeps rows where the pattern has NO matches (checks absence)
FILTER EXISTS { pattern }	Keeps rows where the pattern DOES match (checks presence)
MINUS { pattern }	Subtracts matching rows from the result set

NOT EXISTS and MINUS **usually** give the same result, but they can differ with unbound variables.

Example:

Find Elements with No Colour

```
SELECT ?name ?symbol WHERE {
  ?e prt:name ?name ; prt:symbol ?symbol ; prt:atomicNumber ?n .
  FILTER NOT EXISTS { ?e prt:color ?color }
}
```

Returns only elements that have **no colour property at all**. The NOT EXISTS checks that no colour triple exists for each element.

8 UNION – Combining Alternative Patterns

UNION = OR

UNION combines results from **two or more alternative** patterns. If a match satisfies *either* pattern, it's included.

Analogy: “Show me elements that are red OR blue” – UNION gives you both sets.

```
{ ?e prt:color "red" } UNION { ?e prt:color "blue" }
```

UNION vs OPTIONAL – Key Difference

- **UNION** = “match this OR that” – combines *alternative* patterns
- **OPTIONAL** = “match this, AND *try* to match that too” – adds extra data without excluding base results (LEFT JOIN)

UNION can change which base results you get. OPTIONAL never removes base results.

9 Query Modifiers: ORDER BY, LIMIT, OFFSET

Modifiers are Testable

Know ORDER BY (ascending default, DESC for descending), LIMIT, OFFSET, and how to combine them for pagination.

Modifier	What It Does	SQL Equivalent
ORDER BY ?x	Sort results ascending (A→Z, 0→9) by default	ORDER BY x
ORDER BY DESC(?x)	Sort results descending (Z→A, 9→0)	ORDER BY x DESC
LIMIT n	Return at most n rows	LIMIT n
OFFSET n	Skip the first n rows	OFFSET n

Example:

Find the 5 Heaviest Elements

```
SELECT ?name ?weight WHERE {
  ?e prt:name ?name ; prt:atomicWeight ?weight .
}
ORDER BY DESC(?weight)
LIMIT 5
```

DESC(?weight) sorts from heaviest to lightest. LIMIT 5 takes only the top 5.

Pagination Formula To show results 21–30: LIMIT 10 OFFSET 20

- OFFSET 20 = skip first 20 results
- LIMIT 10 = return next 10 results
- Combined: results 21 through 30

10 GROUP BY, HAVING, and Aggregate Functions

Aggregation is Important

Know GROUP BY, HAVING, the difference between FILTER and HAVING, and the aggregate functions: COUNT, SUM, AVG, MIN, MAX, SAMPLE, GROUP_CONCAT.

GROUP BY = Create Groups for Aggregation

GROUP BY groups rows that share the same value for a variable. You then apply **aggregate functions** to compute values per group.

Analogy: “Count how many students are in each year group” – GROUP BY year, COUNT students.

Function	What It Does
COUNT(?x)	Count items in each group
SUM(?x)	Total of numeric values
AVG(?x)	Average of numeric values
MIN(?x) / MAX(?x)	Smallest / largest value
SAMPLE(?x)	Return one random item from the group
GROUP_CONCAT(?x; separator=", ")	Join all values into one comma-separated string

Example:

Count Elements Per State

```
SELECT ?state (COUNT(?symbol) AS ?num)
WHERE {
  ?e prt:name ?symbol ; prt:standardState ?state .
}
GROUP BY ?state
ORDER BY ?num
```

Step by step:

1. WHERE matches elements with name and state
2. GROUP BY ?state groups by state (gas, liquid, solid...)
3. COUNT(?symbol) counts elements per group
4. ORDER BY ?num sorts by count ascending

HAVING = Filter AFTER Grouping

HAVING applies a condition **after** groups have been created.
Only groups that satisfy the HAVING condition are returned.

Example:

Only States with More Than 10 Elements

```
SELECT ?state (COUNT(?symbol) AS ?num)
WHERE {
  ?e prt:name ?symbol ; prt:standardState ?state .
}
GROUP BY ?state
HAVING (?num > 10)
ORDER BY ?num
```

HAVING filters out groups with 10 or fewer elements.

FILTER vs HAVING – Critical Difference

	FILTER	HAVING
When?	BEFORE grouping	AFTER GROUP BY
Operates on	Individual rows	Aggregated groups
Requires GROUP BY?	No	Yes
Example	FILTER(?age > 18)	HAVING(COUNT(*) > 5)

Memory trick: FILTER = row-level filtering. HAVING = group-level filtering.

11 SPARQL 1.1 Features (2013)

Property Paths and Subqueries

Know property paths (+, *, ?), subqueries, and the four SPARQL 1.1 additions.

11.1 Property Paths – Transitive Traversal

Property Path Operators

Property paths let you follow chains of relationships:

Operator	Meaning	Example
+	One or more hops	foaf:knows+ = 1, 2, 3, ... hops
*	Zero or more hops	foaf:knows* = 0, 1, 2, ... hops
?	Zero or one hop	foaf:knows? = 0 or 1 hop

Example:

Transitive Query – Find All Connections **Data:** Alice knows Bob. Bob knows Charlie. Charlie knows Dana.

```
SELECT ?connection WHERE {
  :Alice foaf:knows+ ?connection .
}
```

Result: Bob, Charlie, Dana (all reachable via 1+ hops).

If we used `foaf:knows*`, Alice herself would also appear (0 hops = the starting node).

11.2 Subqueries

Subqueries = Query Inside a Query

A subquery nests a SELECT inside the WHERE clause of an outer query. Useful for computing aggregates and using them as filters.

Example:

Find People Older Than Average

```
SELECT ?person WHERE {
  ?person foaf:name ?name ;
          foaf:age ?age .
  FILTER(?age > (
    SELECT (AVG(?a) AS ?avg) WHERE { ?p foaf:age ?a }
  ))
}
```

The inner query calculates the average age. The outer query finds people whose age exceeds it.

11.3 Summary: SPARQL 1.1 Additions

Feature	What It Enables
Negation	FILTER NOT EXISTS, MINUS – find things that don't match
Property Paths	Transitive traversal with +, *, ? operators
Aggregation	GROUP BY, COUNT, SUM, AVG, HAVING
Subqueries	Nested SELECT inside WHERE for complex logic

12 SERVICE – Federated Queries

SERVICE = Query Remote Endpoints

SERVICE sends part of your query to a **remote** SPARQL endpoint. This lets you combine data from your local triplestore with external data sources in a single query.

Analogy: You're writing an essay using your own notes (local data) AND asking a friend (remote endpoint) for their notes too.

Example:

Query DBpedia for People Named 'Hadj*'

```
SELECT ?name ?birth_date WHERE {
  SERVICE <http://dbpedia.org/sparql> {
    ?person a foaf:Person ;
            foaf:name ?name ;
            dbpedia:birthDate ?birth_date .
  }
  FILTER REGEX(?name, '^Hadj', 'i')
}
```

The **SERVICE** clause sends the pattern to DBpedia's endpoint. Results are combined with any local patterns.

Tip:

Multiple SERVICE Clauses You can use multiple **SERVICE** clauses to combine data from several remote endpoints in one query. You can also mix local patterns with **SERVICE** clauses for federated querying.

13 Reasoning: Forward and Backward Chaining

Reasoning May Appear on the Test

Know the difference between forward chaining (data-driven) and backward chaining (goal-driven), and be able to walk through examples of each.

Reasoning = Inferring New Knowledge

A **reasoner** applies logical rules to derive new triples that are not explicitly stated. Two main strategies exist:

	Forward Chaining	Backward Chaining
Direction	Facts $\rightarrow$ Conclusions	Conclusions $\rightarrow$ Facts
Also called	Data-driven	Goal-driven
When?	At data load time (once)	At query time (each query)
How?	Apply ALL rules repeatedly until nothing new	Start from what you want to prove, work backward
Speed	Slow setup, fast queries	Fast setup, slower queries
Analogy	Build a complete phone book upfront	Look up numbers on demand

Example:

Worked Example – Rain Scenario **Rules:**

1. Rule 1: If it is raining $\rightarrow$ the ground is wet
2. Rule 2: If the ground is wet $\rightarrow$ the grass is wet

Fact: It is raining.

Example:

Forward Chaining (Facts $\rightarrow$ Conclusions)

1. Start with fact: “It is raining”
2. Apply Rule 1: raining $\rightarrow$ ground is wet. **Add:** “Ground is wet”
3. Apply Rule 2: ground is wet $\rightarrow$ grass is wet. **Add:** “Grass is wet”
4. No more rules can fire $\rightarrow$ **Done!**

All derived facts are now stored and ready for queries.

Example:

Backward Chaining (Goal: “Is the grass wet?”)

1. Goal: “Is the grass wet?”
2. Check Rule 2: To prove “grass is wet”, I need “ground is wet”
3. Check Rule 1: To prove “ground is wet”, I need “it is raining”
4. Check facts: “It is raining” IS a known fact!
5. Chain succeeds $\rightarrow$ **Answer: Yes, the grass is wet**

Trade-off Between Forward and Backward

	Forward	Backward
Upfront cost	HIGH (reasons through everything)	NONE (no pre-computation)
Query speed	FAST (pre-computed)	SLOWER (reasons per query)
Best for	Many queries, stable data	Rare queries, frequently changing data

Neither is “always better” – it depends on how often you query vs how often data changes.

14 Practice Questions – Class Test Style

14.1 Q1: Reading a SPARQL Query

What does this query return? Explain step by step.

```
PREFIX prt: <http://...>
SELECT ?name ?symbol WHERE {
  ?e prt:name ?name ;
    prt:symbol ?symbol ;
    prt:atomicNumber ?n .
  FILTER(?n < 5)
}
ORDER BY ?n
```

Answer:

1. The WHERE clause matches elements that have a name, symbol, AND atomic number
 2. FILTER keeps only elements where atomic number < 5 (i.e., H, He, Li, Be)
 3. SELECT displays only ?name and ?symbol (not ?n or ?e)
 4. ORDER BY ?n sorts results by atomic number ascending
- Result: 4 rows showing Hydrogen/H, Helium/He, Lithium/Li, Beryllium/Be.

14.2 Q2: OPTIONAL vs Required

Query A gives 50 results. Query B gives 60 results. Why?

```
(*@\textcolor{defblue}{Query A:}*) SELECT ?name ?email WHERE { ?p foaf:
  name ?name ; foaf:mbox ?email }
(*@\textcolor{defblue}{Query B:}*) SELECT ?name ?email WHERE { ?p foaf:
  name ?name . OPTIONAL { ?p foaf:mbox ?email } }
```

Answer: Query A **requires** both name AND email – people without email are excluded (50 results). Query B makes email **OPTIONAL** – all 60 people with names are included; 10 of them have blank emails.

14.3 Q3: Identify the Query Type

Match each scenario to the correct query type:

Scenario	Answer
“Does an element with symbol ‘Zz’ exist?”	ASK
“List all element names and atomic numbers”	SELECT
“Show me everything about gold”	DESCRIBE
“Create new triples using rdfs:label from prt:name”	CONSTRUCT

14.4 Q4: FILTER vs HAVING

Explain why FILTER and HAVING cannot be swapped:

```
(*@\textcolor{defblue}{Query A:}*) SELECT ?state (COUNT(?e) AS ?num) WHERE
{
  ?e prt:standardState ?state ; prt:atomicNumber ?n .
  FILTER(?n > 50)
} GROUP BY ?state HAVING(?num > 3)
```

Answer:

- **FILTER(?n > 50)** runs **first** – removes individual elements with atomic number ≤ 50
- **GROUP BY ?state** – groups remaining elements by state

• **HAVING(?num > 3)** runs **after grouping** – removes states with 3 or fewer elements
FILTER works on individual rows **BEFORE** grouping. **HAVING** works on groups **AFTER GROUP BY**.

14.5 Q5: Property Paths

Data: A knows B. B knows C. C knows D.

What does each query return?

Query	Result
:A foaf:knows ?x	B only (exactly 1 hop)
:A foaf:knows+ ?x	B, C, D (1 or more hops)
:A foaf:knows* ?x	A, B, C, D (0 or more hops – includes self)

14.6 Q6: Forward vs Backward Chaining

Given:

- Rule: If X is a Dog $\rightarrow$ X is an Animal
- Fact: Rex is a Dog

Forward chaining: Start from fact “Rex is a Dog”. Apply rule. Derive “Rex is an Animal”. Store it.

Backward chaining: Goal: “Is Rex an Animal?” $\rightarrow$ Rule says: if Rex is a Dog, then yes $\rightarrow$ Check: Rex IS a Dog $\rightarrow$ Confirmed!

14.7 Q7: Writing a SPARQL Query from English

Write a SPARQL query for: “Find the names and colours of all elements that are gases, sorted alphabetically by name.”

Answer:

```
PREFIX prt: <http://www.daml.org/2003/01/periodictable/PeriodicTable#>
SELECT ?name ?color WHERE {
  ?e prt:name ?name ;
    prt:standardState "gas" .
  OPTIONAL { ?e prt:color ?color }
}
ORDER BY ?name
```

Note: OPTIONAL is used for colour because some gases may not have a colour listed.

14.8 Q8: Pagination

Write the modifiers to show results 31–40 of a query, sorted by name descending.

Answer:

```
ORDER BY DESC(?name)
LIMIT 10
OFFSET 30
```

OFFSET 30 skips the first 30. LIMIT 10 takes the next 10 (rows 31–40).

15 Quick Reference Cheat Sheet

SPARQL – Key Concepts for Class Test	
SPARQL	Standard query language for RDF data (like SQL for graphs)
Pattern Matching	SPARQL finds subgraphs matching your described pattern
Variables	Start with ? or \$ – placeholders the engine fills in
PREFIX	Namespace shortcuts (always comes first in query)
SELECT	Returns a table of variable bindings
CONSTRUCT	Returns a new RDF graph (triples) from a template
DESCRIBE	Returns all triples about matched resource(s)
ASK	Returns true/false (existence check)
WHERE { }	Contains triple patterns in Turtle-like syntax
OPTIONAL	Include result even if pattern doesn't match (LEFT JOIN)
FILTER	Boolean condition to restrict results
REGEX	String pattern matching inside FILTER
NOT EXISTS	Keep rows where pattern has no match
UNION	Combine results from alternative patterns (OR)
ORDER BY	Sort results (ASC default, DESC for reverse)
LIMIT / OFFSET	Restrict result count / skip rows (pagination)
GROUP BY	Group rows by variable value for aggregation
HAVING	Filter groups AFTER GROUP BY (not individual rows)
COUNT/SUM/AVG	Aggregate functions (use with GROUP BY)
GROUP_CONCAT	Join values into comma-separated string
Property paths	+ (1+ hops), * (0+ hops), ? (0 or 1 hop)
Subqueries	Nested SELECT inside WHERE for complex logic
SERVICE	Send query part to a remote SPARQL endpoint
Triplestore	Database for storing/querying RDF triples
SPARQL endpoint	Web service accepting SPARQL queries over HTTP
Fuseki	Jena-based SPARQL server with web interface
ETL	Extract, Transform, Load – pipeline for populating triplestores
Forward chaining	Data-driven: facts → conclusions (upfront, fast queries)
Backward chaining	Goal-driven: conclusions → facts (on-demand, slower queries)

Top 10 Exam Traps for SPARQL

1. SPARQL PREFIX has **no @** and **no trailing dot** (unlike Turtle's @prefix)
2. **FILTER** is for rows, **HAVING** is for groups – they are NOT interchangeable
3. **OPTIONAL** = LEFT JOIN. Without it, missing data excludes the entire row
4. **ASK** returns **only true/false** – NOT the matching data
5. **CONSTRUCT** and **DESCRIBE** return **graphs** (triples), not tables
6. **ORDER BY** defaults to **ascending** – use DESC() for descending
7. **foaf:knows+** = 1+ hops; **foaf:knows*** = 0+ hops (includes starting node!)
8. **BETWEEN** does **not exist** in SPARQL – use **FILTER(?x > a && ?x < b)**

9. TOP does **not exist** in SPARQL – use ORDER BY DESC() LIMIT n
10. Forward chaining = slow setup / fast queries. Backward = fast setup / slow queries.

End of Week 4 Study Notes

F20BD/F21BD – Big Data Management | Heriot-Watt University

Good luck with your class test and final exam!